FAULTLINE FRONTIER

Feature Development Roadmap | Noir Forge

Planning horizon: 10–12 weeks • Assumption: one developer with AI-assisted development • Scope: six agreed product features only

Product objective

Evolve the standalone HTML tutorial into an interactive Python backend engineering academy. The core loop is Learn → Experiment → Code & Test → Track Mastery. This plan excludes unrelated Noir Forge infrastructure, marketing, and monetization.

Phased delivery plan

1 | Weeks 1–2 ● Interactive Lesson Engine

React lesson navigation, reusable content schemas, code samples, simulations, quizzes and persistent local progress.

Deliverable: Six migrated lessons with responsive UI and automated smoke tests.

2 | Weeks 3–5 ● The Proving Grounds

Monaco code editor, FastAPI evaluation API, isolated Python execution, pytest grading, timeouts and actionable feedback.

Deliverable: Working coding challenges, securely evaluated.

3 | Weeks 6–7 ● The Failure Range

Deterministic simulations of timeouts, retries, duplicate messages, circuit-breaker recovery and worker failures.

Deliverable: Five repeatable fault-injection labs.

4 | Week 8 ● Bounty Board

Daily challenges, prerequisites, difficulty progression, completion rewards and assignment history.

Deliverable: Daily challenge experience with repeatable scheduling.

5 | Week 9 ● Marshal's Log

Persist attempts, outcomes, concept mastery, mistakes and recommended practice.

Deliverable: Personal learning dashboard that survives sessions.

6 | Weeks 10–12 ● Frontier Expeditions

Guided multi-stage project combining FastAPI, concurrency, database correctness and failure recovery.

Deliverable: One end-to-end engineering expedition with automated verification.

Proposed architecture

Frontend: React, TypeScript, Tailwind CSS, Monaco Editor

API: Python 3.12+, FastAPI

Evaluation: pytest inside disposable, restricted execution sandboxes

Persistence: PostgreSQL; local storage only for early prototype progress

Queue: Redis-backed background jobs when execution volume warrants it

Quality: pytest, Vitest, Playwright; containerized deployment

Security gate: never run learner-submitted Python inside the FastAPI process. Use unprivileged isolated sandboxes with resource limits, network restrictions, execution timeouts, and no application secrets or database access.

MVP boundaries

Include: six existing lessons, five fault scenarios, roughly twelve coding challenges, daily challenge selection, basic progress history, and one guided expedition. Defer: multiplayer, AI tutors, elaborate reward economies, subscriptions, and community challenge marketplaces.

Release gates

- **Lesson engine:** All six lessons functional on desktop/mobile; progress retained.
- **Proving Grounds:** Isolated execution, accurate grading, enforced timeouts and safe error feedback.
- **Failure Range:** Every scenario repeatable with documented expected outcomes.
- **Bounty Board:** Daily selection, completion, and history consistent.
- **Marshal's Log:** Attempts and mastery survive sessions.
- **Frontier Expeditions:** Multi-stage project passes automated verification.

First sprint backlog

- Create faultline-frontier repository and initialize React + TypeScript + Tailwind + Vitest.
- Define lesson, simulation, quiz and coding-challenge schemas.
- Migrate all six existing lessons into structured content.
- Build lesson navigation, progress persistence and responsive layout.
- Add Playwright smoke tests and accessibility checks.
- Document the future code-runner API contract and trust boundaries.

Sprint definition of done: a learner can complete any lesson, run its simulation, answer its quiz, refresh the page and retain completion state.

Sequencing principle

Build the lesson engine first, secure Python execution second, and progression features afterward. Validate educational usefulness before expanding platform complexity.